

ESP32 Web Dashboard for Live Sensor Monitoring and Control

1. Abstract

This project presents the design and implementation of a web-based dashboard using an ESP32 microcontroller to monitor sensor data and control a digital output device remotely. The system uses a **DHT11 temperature and humidity sensor** to measure environmental conditions and displays the data on a web interface. Users can toggle an LED or relay through the dashboard. The system demonstrates real-time IoT monitoring and control using WiFi-enabled embedded systems.

2. Introduction

The Internet of Things (IoT) enables remote monitoring and control of devices over the internet. The ESP32 microcontroller, with built-in WiFi capability, is widely used for such applications.

In this project, the ESP32 hosts a web server that displays real-time sensor data and allows user interaction through a web interface. This eliminates the need for external applications and provides a simple, browser-based solution.

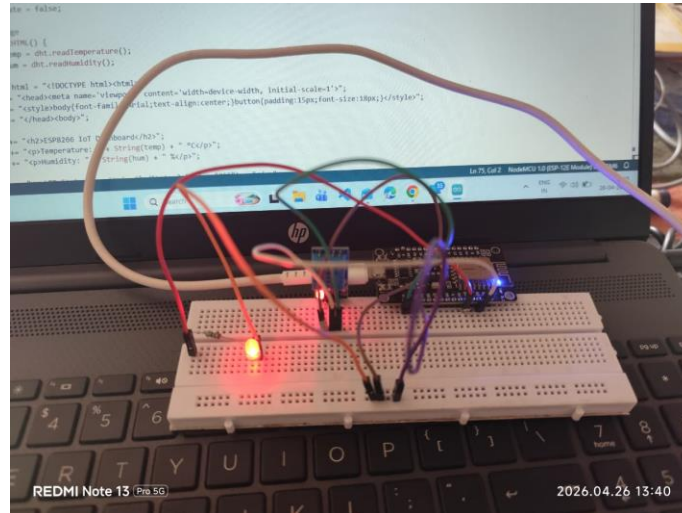
3. Objectives

- To design a web-based dashboard using ESP32
- To display real-time temperature and humidity data
- To control an LED/relay remotely via web interface
- To implement IoT-based monitoring system

4. Components Required

- ESP32 Development Board
- DHT11 temperature and humidity sensor (or DHT22)
- LED / Relay Module
- Resistor (220 Ω)
- Connecting wires
- WiFi network

5. System Connection



6. Working Principle

◆ Sensor Data Acquisition

The DHT11 sensor measures temperature and humidity and sends digital data to the ESP32.

◆ Web Server

The ESP32 acts as a web server using WiFi and serves an HTML page to clients.

◆ Dashboard Interface

- HTML and JavaScript create the user interface
- Data is updated periodically using HTTP requests

◆ Device Control

- A button on the dashboard sends a request to ESP32
- ESP32 toggles LED/relay accordingly

7. ESP32 Code

```
#include <WiFi.h>
#include <WebServer.h>
#include "DHT.h"

#define DHTPIN 4
#define DHTTYPE DHT11
#define LED_PIN 2

const char* ssid = "YOUR_WIFI_NAME";
const char* password = "YOUR_WIFI_PASSWORD";

DHT dht(DHTPIN, DHTTYPE);
WebServer server(80);

bool ledState = false;

String webpage = R"rawliteral(
<!DOCTYPE html>
<html>
<head>
<title>ESP32 Dashboard</title>
</head>
<body style="text-align:center;">
<h2>ESP32 Web Dashboard</h2>
<p>Temperature: <span id="temp">--</span> °C</p>
<p>Humidity: <span id="hum">--</span> %</p>
<button onclick="toggleLED()">Toggle LED</button>
<p>Status: <span id="status">OFF</span></p>

<script>
function fetchData(){
  fetch('/data')
    .then(res=>res.json())
    .then(data=>{
document.getElementById('temp').innerHTML=data.temp;
  document.getElementById('hum').innerHTML=data.hum;
  document.getElementById('status').innerHTML=data.led;
    });
}
```

```

}

function toggleLED(){
  fetch('/toggle');
}

setInterval(fetchData,2000);
</script>
</body>
</html>
)rawliteral";

void handleRoot() {
  server.send(200, "text/html", webpage);
}

void handleData() {
  float temp = dht.readTemperature();
  float hum = dht.readHumidity();

  String json = "{";
  json += "\"temp\":" + String(temp) + ",";
  json += "\"hum\":" + String(hum) + ",";
  json += "\"led\":" + String(ledState ? "ON":"OFF") + "\"";
  json += "}";

  server.send(200, "application/json", json);
}

void handleToggle() {
  ledState = !ledState;
  digitalWrite(LED_PIN, ledState);
  server.send(200, "text/plain", "OK");
}

void setup() {
  Serial.begin(115200);
  pinMode(LED_PIN, OUTPUT);

  dht.begin();

```

```
WiFi.begin(ssid, password);
while (WiFi.status() != WL_CONNECTED) {
  delay(500);
}

server.on("/", handleRoot);
server.on("/data", handleData);
server.on("/toggle", handleToggle);

server.begin();
}

void loop() {
  server.handleClient();
}
```

8. Experimental Results

Parameter	Observed Value
Temperature	25–35 °C
Humidity	40–70 %
LED Control	ON/OFF via browser

9. Observations

- Real-time data updates every 2 seconds
- Stable WiFi connection required
- LED responds instantly to user input
- System works reliably in local network

10. Advantages

- Wireless monitoring and control
- No external app required
- Low cost and easy implementation
- Real-time data visualization

11. Applications

- Smart home automation
- Environmental monitoring
- Industrial IoT systems
- Remote device control

12. Limitations

- Limited to local network (no internet access)
- Sensor accuracy is moderate
- Dependent on WiFi stability

13. Future Enhancements

- Cloud integration (ThingSpeak, Blynk)
- Mobile app interface
- Graphical data visualization
- Multiple sensor integration
- Security (password authentication)

14. Conclusion

The ESP32 web dashboard successfully demonstrates real-time monitoring and control using IoT technology. The system efficiently displays sensor data and allows remote operation of devices, making it suitable for smart automation applications.